

Q1. Student Grade Book Application

CODE

```
data = {
    "Alice": [80,85,90,75,95],
    "Bob": [70,75,80,65,85],
    "Charlie": [90,92,88,95,91],
    "David": [60,65,70,55,68],
    "Eva": [52,58,62,55,70]
}

def get_grade(avg):
    if avg ≥ 90: return "O"
    if avg ≥ 80: return "A+"
    if avg ≥ 70: return "A"
    if avg ≥ 60: return "B+"
    if avg ≥ 50: return "B"
    return "F"

results = {
    name: (sum(marks), sum(marks)/5, get_grade(sum(marks)/5))
    for name, marks in data.items()
}

topper = max(results, key=lambda k: results[k][1])

sub_means = [sum(data[name][i] for name in data)/len(data) for i in range(5)]
best_sub = sub_means.index(max(sub_means)) + 1

print(f"{'Name':<10} {'Total':<6} {'Average':<8} {'Grade'}")
for name, (tot,avg,grade) in sorted(results.items(), key=lambda x: x[1][1], reverse=True):
    print(f"{name:<10} {tot:<6} {avg:<.2f} {grade}")
print(f"\nClass Topper: {topper}")
print(f"Subject with Highest Mean: Subject {best_sub}")
```

OUTPUT

(no output)

Q2. Debugging and Rewriting

CODE

```
accounts = {}
def create_account(acc_no, name, balance):
    accounts[acc_no] = {'name':name, 'balance':balance, 'txns':[]}
def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('Deposit',amount))

def withdraw(acc_no,amount):
    if acc_no in accounts:
        if accounts[acc_no]['balance'] ≥ amount:
            accounts[acc_no]['txns'].append(('Withdraw',amount))
        else:
            print('Insufficient balance')

def mini_statement(acc_no):
    if acc_no in accounts:
        for t in accounts[acc_no]['txns'][-5:]:
            print(t)

create_account(1001, 'Ravi',10000)
deposit(1001,5000)
withdraw(1001,5000)
withdraw(1001,20000)
mini_statement(1001)
print('Balance:', accounts[1001]['balance'])
```

OUTPUT

(no output)